Comandos de interés:

1. **docker ps** → muestra todos los contenedores levantados que estén en funcionamiento.

2. **docker ps -a** →muestra todos los contenedores levantados o no.

3. **docker rm [nombre del contenedor o ID]** → borra los contenedores creados.

4. **docker run [nombre de la imagen]** →crea y ejecuta instantáneamente el contenedor.

5. **docker create [nombre de la imagen]** →crea el contenedor pero no lo ejecuta, para hacerlo se necesita hacer un “docker start”.

6. **docker stop [nombre del contenedor o ID]** → detiene los contenedores creados sin eliminarlos.

7. **docker start [nombre del contenedor o ID]** → inicia o ejecuta un contenedor creado o parado.

8. **docker images** → para ver las imágenes/SO y sus versiones que administra docker.

9. docker search --filter=is-official=true ubuntu → para buscar en terminal las imágenes oficiales en este caso de “ubuntu”.

10. **docker pull [nombre de la imagen]** → para descargar las imágenes deseadas (mysql, ubuntu, httpd… etc).

Descargar imagenes/ISOS para contenedores

1. docker pull ubuntu:latest → descarga última versión disponible de Ubuntu proveniente de docker hub. (tanto con :lastest como sin él te descarga la última versión) .

2. Si queremos otra version hacemos: docker pull mysql:5.7

Ejecutar o abrir contenedor

1. Levantar un contenedor sin meternos dentro sería:
   docker run [nombre de la imagen]

2. Levantar un contenedor metiéndonos dentro con una terminal iterativa es:
   docker run -it ubuntu /bin/bash

Abrir contenedor con nombre e imagen

1. Una vez levantado un contenedor vamos a querer entrar o acceder a una de sus terminales con un nombre (en este caso apuntes).
   docker exec [opciones] nombre_contenedor orden [argumentos]
   docker run -dit --name apuntes ubuntu
   docker exec -it web /bin/bash

2. Podemos cambiarles los puertos cuando levantemos el contenedor:
   docker run -dit --name apuntes -p 8080:80 ubuntu

Enviar ficheros desde la máquina host/real al contenedor

1. Creas un fichero en la máquina real. Ejemplo: web3.html
   touch web3.html

2. Ejecutamos un cp para enviar el fichero del host al contenedor:
   docker cp web3.html web:/usr/local/apache2/htdocs/

3. En el navegador de la máquina virtual ponemos:
   127.0.0.1:8080/web3.html

Montar enlace o directorio

1. En la máquina real/host creas una carpeta.
   Ejemplo: mi-web →mkdir -p ./mi-web

2. Introducimos un echo o código que quieras:
   echo "<h1>Hola desde ASIR2 en la clase ASO</h1>" > ./mi-web/pruebacarpeta.html

3. Creo o levanto un contenedor en segundo plano (-d) con puerto (8282) y que conecte (-v) el directorio anteriormente creado a una ruta determinada.
   docker run -d -- name web-bind -p 8282:80 -v ./mi-web:/usr/local/apache2/htdocs/httpd:lastes

4. En el navegador de la máquina virtual ponemos:
   127.0.0.1:8282

Montar volúmenes

1. Creas el volumen con nombre web_content:
   docker volume create web_content

2. Creo o levanto un contenedor en segundo plano (-d) con puerto (8383) y que conecte (-v) el volumen anteriormente creado a una ruta determinada.
   docker run -d --name web-vol -p 8383:80 -v web_content:/usr/local/apache2/htdocs httpd:latest

3. Ahora para introducir un fichero al volumen ya que este está vacío, En lugar de arrancar el contenedor de Apache (httpd) y entrar con docker exec para crear el archivo a mano, lo que hacemos es usar un contenedor desechable (Alpine) para hacer esa tarea de forma rápida y automatizada.

   docker run --rm -v web_content:/data alpine sh -c "echo '<h1>Contenido persistente de Angel Lara - asignatura ASO</h1>' > /data/content.html"

   docker run: Arranca un nuevo contenedor.

   --rm: Esta es la clave. Le dice a Docker que destruya el contenedor en el momento en que termine de ejecutar la orden. Es un contenedor "de usar y tirar". No queremos que se quede ahí ocupando espacio.

   -v web_content:/data: Conecta el volumen llamado web_content (que creaste antes) a la carpeta /data dentro de este contenedor temporal.

   alpine: Es la imagen que usamos. ¿Por qué Alpine? Porque es una distribución de Linux ultraligera (pesa apenas unos megas). Si usáramos una imagen como ubuntu (que pesa mucho más) solo para escribir un archivo de texto, sería una pérdida de tiempo y recursos. Alpine es perfecta para tareas rápidas en segundo plano.

   sh -c "...": Aquí le decimos a Alpine qué debe hacer una vez que arranque. Le pedimos que abra una consola (sh) y ejecute el comando que está entre comillas (-c).
   echo '<h1>Contenido persistente</h1>' > /data/index.html: Este es el comando real. Escribe el texto HTML y lo guarda (con el símbolo >) en el archivo index.html dentro de la carpeta /data. Como esa carpeta /data está conectada al volumen web_content, el archivo se guarda realmente el volumen.

Listar parámetros de contenedores

- En este caso hablaremos de los servicios como apache y nginx
    1. **docker inspect [nombre o ID del contenedor levantado]** →aquí podremos ver el id del contenedor, versión de la imagen o servicio, nombre, puertos que usa, IP, Gateway y MAC.

    2. **docker stats [nombre o ID del contenedor levantado]** →aquí podremos ver el consumo que ejerce el contenedor en tiempo real.

    3. **docker top [nombre o ID del contenedor levantado]** → aquí podremos ver que usuarios y procesos se están ejecutando.

DockerFile

- Un DockerFile es un fichero, que representa una plantilla, receta o instrucciones.

- Sirve para levantar un contenedor sin tener que configurarlo por “COMANDOS”.

- Instrucciones mínimas de un DockerFile:
    1. ***FROM httpd:2.4*** → Indicamos que nuestro contenedor tendrá una imagen apache http server en version 2.4

    2. ***COPY ./ /usr/local/apache2/htdocs/*** → Copia todos los archivos del directorio actual de nuestro hosts a la carpeta pública de Apache.

    3. ***EXPOSE 80*** → Indica que el contenedor utiliza el puerto 80.
- Construimos la imagen con:
  docker build -t (nombre de una imagen) .
  `docker build` → construye una imagen.
  `-t` → asigna un nombre o etiqueta.
  `mi-web` → nombre de la imagen.
  `.` → indica que el contexto de construcción es el directorio actual.

- Creamos y arrancamos el contenedor con DockerFile.
  docker run -d --name web -p 8080:80 mi-web

Instrucciones para construir y personalizar la imagen

Instrucción	¿Qué permite hacer?	Ejemplo
FROM	Elegir la imagen base.	`FROM ubuntu:24.04`
LABEL	Añadir información y metadatos.	`LABEL maintainer="admin"`
RUN	Ejecutar comandos durante la construcción de la imagen.	`RUN apt update`
COPY	Copiar archivos del equipo a la imagen.	`COPY index.html /var/www/html/`

Expose	Indica qué puerto utiliza o espera recibir conexiones el servicio dentro del contenedor.	`EXPOSE 80`
		`EXPOSE 22`
		`EXPOSE 80 443`

ADD	Copiar archivos y también realizar algunas operaciones adicionales, como extraer archivos comprimidos locales.	`ADD web.tar /var/www/html/`
WORKDIR	Establecer el directorio de trabajo.	`WORKDIR /var/www/html`
ENV	Definir variables de entorno permanentes de la imagen.	`ENV APP_ENV=production`
ARG	Definir variables disponibles durante la construcción.	`ARG VERSION=1.0`

1. LABEL →Permite añadir información descriptiva a la imagen. Sirve para identificar al creador, la finalidad o la versión de una imagen. No modifica el funcionamiento del contenedor.
   LABEL maintainer="Administrador"
   LABEL description="Servidor web Apache"
   LABEL version="1.0"

2. RUN → Ejecuta comandos durante la construcción de la imagen.
   FROM ubuntu:24.04
   RUN apt update && apt install -y apache2

Docker Compose – Relacionar contenedores

1. Creamos un directorio → ***mkdir -p docker-compose-practica***
2. Dentro de docker-compose-practica creamos los ficheros:
   - ***DockerFile***

```
alp@alp-VM:~/docker-compose-practica$ cat Dockerfile
FROM alpine:3.21

LABEL maintainer="ALP@asir.local"
LABEL description="Apache + PHP + MySQL con Docker Compose"

# Instalar Apache, PHP y las extensiones necesarias
RUN apk add --no-cache \
    apache2 \
    php83 \
    php83-apache2 \
    php83-pdo \
    php83-pdo_mysql

# Configurar Apache para ejecutar PHP
RUN echo "DirectoryIndex index.php index.html web-compose.php" >> /etc/apache2/httpd.conf

# Copiar los archivos de la página web
COPY web/ /var/www/localhost/htdocs/

# Puerto del servidor web
EXPOSE 80

# Mantener Apache en primer plano
CMD ["httpd", "-D", "FOREGROUND"]
```

   - ***docker-compose.yml***

```
alp@alp-VM:~/docker-compose-practica$ cat docker-compose.yml
services:

  web:
    build: .
    container_name: apache-php
    ports:
      - "8080:80"
    volumes:
      - ./web:/var/www/localhost/htdocs/
    depends_on:
      - db
    networks:
      - red-interna

  db:
    image: mysql:8.0
    container_name: mysql-bbdd
    environment:
      MYSQL_ROOT_PASSWORD: asir123
      MYSQL_DATABASE: instituto
    volumes:
      - datos_mysql:/var/lib/mysql
    networks:
      - red-interna

volumes:
  datos_mysql:

networks:
  red-interna:
    driver: bridge
```

3. Dentro de ***/docker-compose-practica*** creamos el directorio **web**:
   - En el directorio web alojaremos los ficheros ".php" o ".html"

```
alp@alp-VM:~/docker-compose-practica/web$ ls
web-compose.php  web-info.php
```

4. Lanzamiento de docker compose:
   - Tenemos que estar dentro de ***/docker-compose-practica***
   - Ejecutamos el comando: ***docker compose up -d –build***